Introducción a Python para Programadores de C

1/3 – Sintaxis, tipos, control de flujo y funciones

Informática y Programación

UNCuyo – Facultad de Ingeniería

9 de septiembre de 2026

Índice de Contenidos

1. 1. Sintaxis Fundamental
2. 2. Variables y Tipos de Datos
3. 3. Estructuras de Control
4. 4. Entrada de Datos desde Teclado
5. 5. Funciones y Parámetros
6. 6. Resumen y Conclusiones

Por qué cambiamos de lenguaje ahora

No es un lenguaje nuevo desde cero

Ya conocés variables, tipos, condicionales, bucles y funciones — los mismos **conceptos** de siempre, pero sin compilar y con menos sintaxis. Esta serie va a apoyarse todo el tiempo en lo que ya sabés de C para aprender Python por **contraste**, no por repetición.

Esta es la presentación 1 de 3

1. **Fundamentos** (ésta): sintaxis, tipos, control, E/S, funciones.
2. **Estructuras de datos**: listas, tuplas y diccionarios.
3. **Cadenas, archivos y funciones avanzadas**.

Indice de Contenidos

1. 1. Sintaxis Fundamental
2. 2. Variables y Tipos de Datos
3. 3. Estructuras de Control
4. 4. Entrada de Datos desde Teclado
5. 5. Funciones y Parámetros
6. 6. Resumen y Conclusiones

¡Hola, Mundo!

Puntos Clave

- Python es un lenguaje interpretado, no requiere compilación explícita.
- No hay función `main()` obligatoria.
- El punto y coma `;` es opcional y no se usa.
- Los bloques de código se definen por **indentación**, no por llaves `{}`.

C

```c
#include <stdio.h>

int main() {
    printf("Hola, Mundo!\n");
    return 0;
}
```

Python

```python
print("Hola, Mundo!")
```

Cuidado: la indentación es sintaxis

No es solo prolijidad

En C, la indentación es una convención visual — el compilador no la mira, solo le importan las llaves. En Python, la indentación **define** qué líneas pertenecen a qué bloque. Mezclar espacios y tabs, o indentar mal, es un error de sintaxis real (`IndentationError`).

```
if 5 > 3:
    print("Se ejecuta")    # pertenece al if (4 espacios)
      print("Error!")      # IndentationError: indentado de mas
```

Indice de Contenidos

1. 1. Sintaxis Fundamental
2. 2. Variables y Tipos de Datos
3. 3. Estructuras de Control
4. 4. Entrada de Datos desde Teclado
5. 5. Funciones y Parámetros
6. 6. Resumen y Conclusiones

Variables y Tipos de Datos

Diferencia Principal: Tipado

- **C es de tipado estático:** Debes declarar el tipo de la variable antes de usarla (`int`, `float`, etc.).
- **Python es de tipado dinámico:** El tipo se infiere automáticamente en el momento de la asignación.

C

```c
int cantidad = 10;
float precio = 95.50;
```

Python

```python
# La variable toma el tipo
# del valor asignado
cantidad = 10
precio = 95.50

# Podemos verificar el tipo
# type(cantidad) -> <class 'int'>
```

Cuidado: dinámico no es "sin tipo"

La variable puede cambiar de tipo, el valor no

"Dinámico" significa que el mismo *nombre* puede referirse a valores de distinto tipo en distintos momentos — no que Python ignore los tipos. Sigue siendo **fuertemente tipado**: no suma un `int` y un `str` sin conversión explícita, igual que C.

```
x = 10          # x es int
x = "diez"      # ahora x es str (valido, x cambio de tipo)

edad = 20
print("Tengo " + edad)    # TypeError: no se puede sumar str + int
print("Tengo " + str(edad))  # correcto: conversion explicita
```

Indice de Contenidos

1. 1. Sintaxis Fundamental
2. 2. Variables y Tipos de Datos
3. 3. Estructuras de Control
4. 4. Entrada de Datos desde Teclado
5. 5. Funciones y Parámetros
6. 6. Resumen y Conclusiones

Condicionales: if / elif / else

Lógica de Ejecución

La estructura ejecuta **únicamente el primer bloque** cuya condición sea verdadera (`true`) y luego sale por completo, ignorando el resto.

C

```c
int nota = 75;

if (nota >= 90) {
    printf("Excelente\n");
} else if (nota >= 60) {
    printf("Aprobado\n");
} else {
    printf("Reprobado\n");
}
// Salida: Aprobado
```

Python

```python
nota = 75

if nota >= 90:
    print("Excelente")
elif nota >= 60:
    print("Aprobado")
else:
    print("Reprobado")
# Salida: Aprobado
```

Equivalente a Switch

Simulando un Switch

Python no tiene una estructura `switch-case` tradicional. El comportamiento idéntico se logra con una cadena `if/elif/else`.

C

```c
int dia = 2;
switch (dia) {
    case 1: printf("Lunes\n"); break;
    case 2: printf("Martes\n"); break;
    default: printf("Otro dia\n");
}
```

Python

```python
dia = 2
if dia == 1:
    print("Lunes")
elif dia == 2:
    print("Martes")
else:
    print("Otro dia")
```

Bucles: while

Sintaxis Similar

La lógica del `while` es idéntica. Cambian los detalles: sin paréntesis en la condición y sin el operador `i++`.

C

```c
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

Python

```python
i = 0
while i < 5:
    print(i)
    i += 1
```

Python no tiene `i++` ni `i--`: se usa `i += 1`.

Bucles: for

Filosofía Diferente

El `for` de C es un contador. El `for` de Python itera sobre secuencias. Para replicar el comportamiento de C, usamos `range()`.

C (Contador)

```c
int i;
for (i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

Python (Iterador)

```python
# range(5) genera 0, 1, 2, 3, 4
for i in range(5):
    print(i)
```

`range()` con inicio, fin y paso

Igual de flexible que un `for` de C

`range(inicio, fin, paso)` — `fin` nunca se incluye, igual que la condición de corte de un `for` en C.

C

```c
for (int i = 10; i > 0; i--) {
    printf("%d\n", i);
}
```

Python

```python
for i in range(10, 0, -1):
    print(i)
```

Indice de Contenidos

1. 1. Sintaxis Fundamental
2. 2. Variables y Tipos de Datos
3. 3. Estructuras de Control
4. 4. Entrada de Datos desde Teclado
5. 5. Funciones y Parámetros
6. 6. Resumen y Conclusiones

Lectura de Datos: scanf vs input()

Diferencia Clave

- En C usas `scanf()` con especificadores de formato.
- En Python usas `input()`, que **siempre retorna un string**.
- Debes convertir explícitamente a otros tipos (`int()`, `float()`).

C

```c
char nombre[50];
int edad;

printf("Nombre: ");
scanf("%s", nombre);

printf("Edad: ");
scanf("%d", &edad);

printf("Hola %s\n", nombre);
```

Python

```python
# input() retorna string
nombre = input("Nombre: ")

# Convertimos a entero
edad = int(input("Edad: "))

print(f"Hola {nombre}")
```

Lectura de Múltiples Valores

Técnica con split()

Python puede leer varios valores en una línea usando `split()` y desempaquetado.

C

```c
int a, b;

printf("Dos numeros: ");
scanf("%d %d", &a, &b);

printf("Suma: %d\n", a + b);
```

Python

```python
# Leer y separar
entrada = input("Dos numeros: ")
a, b = entrada.split()

# Convertir a enteros
a = int(a)
b = int(b)

print(f"Suma: {a + b}")
```

Indice de Contenidos

1. 1. Sintaxis Fundamental
2. 2. Variables y Tipos de Datos
3. 3. Estructuras de Control
4. 4. Entrada de Datos desde Teclado
5. 5. Funciones y Parámetros
6. 6. Resumen y Conclusiones

Definición de Funciones

Definición Simplificada

En Python se usa la palabra clave `def`. No se declaran tipos de retorno ni de parámetros.

C

```c
int sumar(int a, int b) {
    return a + b;
}

int main() {
    int res = sumar(10, 5);
    // res es 15
    return 0;
}
```

Python

```python
def sumar(a, b):
    return a + b

# --- Llamada ---
res = sumar(10, 5)
# res es 15
```

Paso de Parámetros: Mutabilidad

Punto Clave

- **C pasa copias (por valor).** Para modificar el original, se usan punteros (*).
- **Python pasa referencias a objetos.**
  - Objetos **inmutables** (números, strings) se comportan como paso por valor.
  - Objetos **mutables** (listas, diccionarios) se comportan como paso por referencia.
- El resultado final en el ejemplo que sigue es idéntico en los dos lenguajes.

Paso de Parámetros: mismo resultado en los dos lenguajes

C

```c
void mod(int num, int arr[]) {
    num = 100;    // no cambia
    arr[0] = 99; // SI cambia
}

// mi_numero -> 10
// mi_array[0] -> 99
```

Python

```python
def mod(num, lista):
    num = 100      # no cambia
    lista[0] = 99 # SI cambia

# mi_numero -> 10
# mi_lista[0] -> 99
```

Por qué se parecen tanto: en C, un arreglo pasado como parámetro ya decae a puntero — la dirección se copia por valor. Reasignar `arr` adentro de la función no movería el arreglo del `main`, igual que reasignar `lista` en Python no afecta a la variable de afuera. Lo que sí se comparte, en los dos lenguajes, es lo que hay en esa dirección.

Indice de Contenidos

1. 1. Sintaxis Fundamental
2. 2. Variables y Tipos de Datos
3. 3. Estructuras de Control
4. 4. Entrada de Datos desde Teclado
5. 5. Funciones y Parámetros
6. 6. Resumen y Conclusiones

Tabla Comparativa Rápida

Característica	**Lenguaje C**	**Lenguaje Python**
Tipado	Estático (fuerte)	Dinámico (fuerte)
Compilación	Compilado	Interpretado
Sintaxis	Verbosa, con `;` y `{}`	Concisa, basada en indentación
Manejo de Memoria	Manual (`malloc`, `free`)	Automático (Garbage Collector)
Punteros	Explícitos y fundamentales	No existen de forma directa
Bucle `for`	Basado en un contador	Basado en iteración sobre secuencias

Lo que sigue

Con esto ya podés

Traducir a Python cualquier programa de C que use solo variables, condicionales, bucles y funciones con parámetros simples — la base de los ejercicios 5 y 6 del TP4 (parking, batalla naval).

Presentación 2: Estructuras de Datos

Para modelar los tres pisos del parking, el tablero de la batalla naval, o el inventario de la librería, hace falta más que variables sueltas: **listas, tuplas y diccionarios** — el equivalente Python de los arreglos y `structs` de C. Eso es lo que viene.

¡A programar en Python!